web10: User-Owned Social Data with IAM-Grade Access Contracts at Zero Scan Penalty

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

Social platforms concentrate user data in platform-controlled silos with coarse, platform-defined access: users cannot leave with their data, cannot govern who sees what at fine granularity, and cannot monetize their own audience. Cloud identity-and-access-management (IAM) solved fine-grained, revocable authorization for infrastructure a decade ago, but never exposed that model to end users.

We present **web10**, a deployed data architecture in which every user owns their data and access is governed by fine-grained, revocable, group/role-scoped contracts—bringing IAM primitives to the *front* of a social data platform. Two trust layers gate every request: an *app contract* (which origin may perform which operation on which data service) and a *group role* (which principal may do what, expressed as a per-service permission map). Publicness is not a flag; it is a role grant to a principal class. The entire model is enforced at the query layer over a single columnar store, so fine-grained authorization is an index lookup into small metadata tables and adds *zero scan-time penalty* to the columnar read.

We report initial production measurements from the running system: [**N**] users, [**N**] documents, and [**N**] access decisions per day, with access decisions completing in [**X**] ms at p99. A full evaluation, including a cost comparison against a shared-table baseline with row-level security, appears in the extended version.

1 INTRODUCTION

The dominant social platforms solve engagement extremely well and ownership extremely poorly. A user's posts, messages, and relationships live in the platform's database under the platform's access model. The user can adjust a handful of visibility settings the platform chose to expose; they cannot redefine who a "friend" or a "member" is, cannot contract access per-service, and cannot take their data—let alone their audience— elsewhere. When the platform changes its rules or goes away, the user's data goes with it.

Cloud infrastructure solved the authorization half of this problem long ago. AWS IAM and its descendants give operators fine-grained, role-based, service-scoped, revocable access control with cryptographic token verification. But that model lives at the *infrastructure* boundary: the cloud provider owns the data plane, and the end user is invisible to it. Nobody has brought the IAM model to the *user* boundary of a social platform—where the user, not the platform, is the principal that holds the contracts.

web10 is a social data platform that makes that move. Every user's data lives under their own key in a single columnar store; every access decision is a check against two user-held contract layers; and the social application (feeds, groups, direct messages, discovery) is built entirely from these primitives with no app-specific surface in the node. This paper makes three contributions:

(1) **A two-layer access model for user-owned social data** (§2): an outer *app trust* layer (per-origin, per-service operation grants) and an inner *person trust* layer (group roles as per-service permission maps over three nested principal classes). Both must pass; either can deny.
(2) **A query-layer enforcement design** (§3) in which the role check never touches the large document table. Effective permissions are resolved against small metadata tables, producing a list of cleared group identifiers that the columnar scan consumes as an ordinary filter. Fine-grained control, zero scan-time penalty.
(3) **Initial production measurements** (§4) from the deployed system, with a full evaluation (latency curves, cost model, shared-table baseline) in the extended version.

The remainder of this paper: §2 defines the access model; §3 shows how it is enforced over one columnar table; §4 reports initial numbers; §5 positions web10 against Solid, federated social protocols, and cloud IAM; §6 concludes.

2 THE ACCESS MODEL

2.1 Two trust layers

Every request in web10 names an *app* (the calling origin), a *principal* (the acting user), a *group* G, a *service* S (a data namespace such as `posts` or `comments`), and an *operation* O (e.g. `readAll`, `create`). Authorization asks two independent questions, and *both* must pass:

Layer	Question
App trust (outer wall)	Can this *app* perform O on S?
Person trust (inner wall)	Can this *principal*'s effective role in G grant O on S?

The app contract is *infrastructure trust*: which origins may touch which services, enforced by CORS in the browser and re-checked server-side on every call. The group role is *social trust*: which people may do what inside a group. The split matters because the two layers change at different rates and for different reasons—a user re-points an app's access without leaving any group, and leaves a group without touching any app's access.

2.2 Roles are per-service permission maps

A role is a named bundle of per-service permissions—a map from service to the operations it grants:

```
{ "name": "editor",
  "permissions": {
    "posts":    ["readAll", "create", "updateOwn"],
    "comments": ["readAll"]
  } }
```

The map *is* the scope. `'*'` wildcards all document services; a reserved key `'group'` carries *structural* operations on the group itself (`manageRoles`, `assignRoles`, `deleteGroup`, join/member operations), while *content* operations—including moderation such as `hideAll`—live under the service key, because they act on documents, not on the group's shape.

Two properties keep the model small. First, the role map is the *same shape* as the app-contract permission object: there is one permission language across both trust layers. Second, a person holds *exactly one role* per group; the per-service map makes a single role fully expressive over any (principal, service, operation) matrix, so there is no multi-role stacking to reason about. Roles are generic—a group defines whatever roles fit its purpose (`owner`, `moderator`, `page-curator`, `member`, or `dj`, `listener`)— and the platform never names them.

2.3 Principal classes and effective permissions

Access is granted to *principals*, of which there are three nested classes, broadest to narrowest:

Class	Who
`anyone`	every request, signed in or not
`authenticated`	any valid token (any user, member or not)
`member`	holds a member row in this group

The classes are *reserved keys in the member table*—no separate visibility table exists. A group grants a class a role by storing a member row for the reserved key:

```
group_members:
  (G, 'anyone',        'reader')    -- the group is
       public
  (G, 'authenticated', 'reader')    -- signed-in users
       can read
  (G, 'bob',           'moderator') -- bob moderates
  (G, 'alice',         'owner')     -- alice owns
```

A principal's *effective* permissions in a group are the union of the permission maps of every class they belong to:

$$\begin{aligned}\text{effective}(P, G) \;=\; & \text{grant}(G,\ \texttt{anyone}) \\ \cup\ & \big(P \in \texttt{authenticated}\ ?\ \text{grant}(G,\ \texttt{authenticated}) : \emptyset\big) \\ \cup\ & \big(P \in \texttt{member}(G)\ ?\ \text{grant}(G,\ \text{role}(P)) : \emptyset\big)\end{aligned}$$

The nesting buys a monotonicity invariant for free: a member always sees at least what a signed-in stranger sees, who always sees at least what a signed-out visitor sees. The model cannot express the inversion.

2.4 Publicness is a grant, not a flag

There is no `visibility` column. "Make my group public" is the act of granting `readAll` to `anyone`; "signed-in only" grants it to `authenticated`; "private" grants it to member roles only. Join policy is orthogonal: it controls how a human *becomes* a member (open, request, invite-only) and says nothing about what a non-member bystander can see. A group can be invite-only to join and still publicly readable—the discovery board is exactly that.

This is the sense in which web10 brings IAM to the front: the operations a cloud operator would express as policies—*principal class, service, operation, grant*—are the operations an *end user* expresses about their own data, with the same revocability (a grant is a row; removing it is a tombstone that takes effect on the next read).

3 ENFORCEMENT AT THE QUERY LAYER

3.1 One columnar table

All user content lives in a single ClickHouse table, keyed by `(author_key, doc_id)`:

```
CREATE TABLE documents (
    doc_id          String,
    author_key      String,
    collection_name String,   -- the service: 'posts', '
         comments', ...
    body            String,   -- JSON
    tags            Array(String),
    created_at      DateTime64(3),
    updated_at      DateTime64(3),
    deleted         UInt8 DEFAULT 0
) ENGINE = ReplacingMergeTree(updated_at)
ORDER BY (author_key, doc_id);
```

Updates are inserts with a higher `updated_at` (the engine keeps the latest version); deletes are tombstone rows filtered at read time. Documents attach to groups through a small mapping table, and groups carry their contracts and members in two more small tables. Content is one wide columnar table; *access is a handful of narrow metadata tables.*

3.2 The role check never touches the document table

The read path is the heart of the design:

(1) **Resolve.** Look up the reader's rows in `group_members` (including the reserved principal-class rows) and the role maps in `group_contracts`. These are small, hot, and indexed by their primary keys.
(2) **Compute.** For the requested service, compute the set of group identifiers whose effective role grants `readAll` to the reader.
(3) **Scan.** Run the columnar `documents` scan exactly as it always would, with the cleared-group list as an ordinary filter, unioned with the reader's own `author_key`.

The authorization decision is therefore an index lookup into small metadata tables, not a per-document check inside the scan. The scan stays a scan. This is the "zero scan-time penalty" property: fine-grained, per-group, per-service control costs the same as the coarsest possible filter, because it *is* a filter list computed once per query.

3.3 The same invariant on every path

The enforcement rule is uniform: *no query returns documents for an* `author_key` *the token does not own, unless group membership grants access* (invariant I3). Every read path—group feed, read-by-id, directory search—applies the identical principal-class gate, and the write side is gated the same way, so a bystander cannot attach content to a group they cannot write. A conformance suite of permission-matrix cases (including replayed-exploit denials) enforces the invariants mechanically; the full suite results appear in the extended version.

4 INITIAL EVALUATION

web10 is deployed and in active use. This workshop version reports the measurement plan and initial scale; the extended version adds latency curves, the cost model, and the shared-table baseline comparison.

Scale (initial). [**N**] registered users; [**N**] documents across [**N**] services; [**N**] groups; [**N**] access decisions per day.

Access-decision latency (initial). The resolve+compute phase (metadata lookup + effective-role computation) completes in [**X**] ms at p50 / [**Y**] ms at p99, measured as a fraction of [**Z**]% of total read latency.

Planned (extended version). (1) p50/p95/p99 access-decision latency under sustained load, with the signature-verify / contract-lookup / membership-lookup breakdown; (2) the cost of the per-user key-space model versus a single shared table with row-level security, including storage overhead and the cost growth curve; (3) the same social application's end-to-end flow latencies on web10 versus a conventional backend; and (4) the full permission-matrix conformance results.

5 RELATED WORK

Decentralized social data. Solid [1] gives users personal data stores (Pods) with WebACL-style access control and scoped app tokens; ActivityPub [2] and ATProto [3] federate social activity with user-held datastores. These systems establish that users can own their data, but their authorization is either coarse (container-level ACLs) or federation-trust based, and none makes fine-grained, per-service, role-scoped *group* contracts a first-class, query-enforced primitive with measured scan cost. web10's closest philosophical ancestor is Solid; the differences are storage (one columnar table versus per-Pod LDPC containers), authorization (per-service role maps versus WebACL), and the explicit zero-scan-penalty enforcement point.

Cloud IAM. AWS IAM [4] and the RBAC/ABAC literature [5] define the permission language web10 reuses at the user boundary: principals, roles, service-scoped operations, revocable grants. The move in this paper is directional—the same policy shape, held by the end user about their own data, enforced by the data layer rather than a control plane.

Multi-tenant isolation. The database literature on tenant isolation—schema-per-tenant, database-per-tenant, and row-level security [6]—is the closest systems ancestor. web10's key-space per user is tenant-per-end-user taken to its limit: the tenant is the *end user*, and the isolation is expressed as the query's `author_key` filter plus group-grant list rather than a physical boundary. The cost tradeoff of that choice is measured in the extended version.

6 CONCLUSION

web10 brings the cloud IAM permission model—principal classes, roles, service-scoped operations, revocable grants—to the user boundary of a social data platform, and enforces it at the query layer of a single columnar store so that fine-grained authorization costs an index lookup, not a scan. This workshop version presents the model and initial production measurements; the extended version adds the full evaluation, including the cost comparison against row-level-security baselines.

REFERENCES

[1] T. Berners-Lee. Solid: A distributed social web protocol. *Solid Project / W3C*, 2014–present. https://solidproject.org.
[2] M. Sporny, D. Wilson, R. Smith, and D. Sadok. ActivityPub: A protocol for distributed federated publishing, archiving, and sharing. *IETF Internet-Draft*, 2023.
[3] D. Holman and M. Roth. AT Protocol: A personal data protocol for social applications. *Bluesky Technologies*, 2022–present. https://github.com/bluesky-social/atproto.
[4] Amazon Web Services. AWS Identity and Access Management (IAM) user guide. *Amazon Web Services*, 2025. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html.
[5] R. Sandhu, E. Coyne, H. Feinstein, and C. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.
[6] A. Kemper and S. Lott. Row-level security in PostgreSQL. *PostgreSQL Documentation*, 2024. https://www.postgresql.org/docs/current/ddl-rowsecurity.html.